

Trabajo Práctico Final: Organización de las Computadoras

Joaquín García A.,
Lucas Enedín García,
Sebastián Ayrton Deharbe

joa.garcia@gmail.com,
lucasgarcia2297@gmail.com,
sebasdeharbe@gmail.com

UNL Facultad de Ingeniería y Ciencias Hídricas

13 de noviembre de 2024

Resumen

En este informe se presentará el diseño e implementación de un procesador RISC - V monociclo visto en clase.

Keywords: Microarquitectura, RISC - V, procesador

1. Introducción

EN este trabajo tenemos como objetivo construir un procesador que se basa en la arquitectura de conjunto de instrucciones RISC-V, un ISA libre y de código abierto. Siguiendo la filosofía RISC (conjunto de instrucciones reducido), la arquitectura lo que hace es implementar las instrucciones más utilizadas para optimizar el rendimiento global. Se construyó en Verilog un procesador RISC-V monociclo con la capacidad de ejecutar diversas instrucciones, entre ellas, sumas, restas, instrucciones de carga y almacenamiento de datos en memoria. El informe detalla las implementaciones necesarias para que este procesador pueda ejecutar el código de prueba en lenguaje .ASM que se nos ha proporcionado.

2. Desarrollo

EL sistema propuesto, consta de dos módulos. El módulo de hardware, que es el procesador RV32I construido a partir de las guías prácticas 6 y 7 y de los diagramas y material teórico propuesto por la cátedra que a su vez está dividido en Datapath y módulo memory. Además el sistema consta también de un módulo de software realizado en lenguaje ensamblador en RARS que incluye operaciones aritméticas lógicas simples entre los valores inicializados en los registros.

2.1. Módulo de Hardware:

El diseño y conexión de los módulos en un procesador monociclo de 32 bits es crucial para la ejecución eficiente de instrucciones. Cada uno de estos módulos tiene una función específica que, en conjunto, permite al procesador llevar a cabo operaciones lógicas, aritméticas y de control de flujo en un solo ciclo de reloj.

- **PC:** El contador de programa (PC) lleva la dirección de la próxima ejecución a ejecutar. Este módulo recibe el siguiente PC calculado, junto con la señal de reloj que determina cuando actualizarse.
- **Adder:** El sumador incrementa el valor del PC en 4, ya que cada instrucción ocupa 4 bytes en un sistema de 32 bits. Esto facilita el paso de una instrucción a la siguiente en memoria, pero también permite calcular la nueva dirección en el caso de una instrucción de salto.
- **MUX1:** Este multiplexor selecciona entre dos posibles direcciones para el siguiente PC: una dirección secuencial ($PC + 4$) y una dirección de salto (calculada en base a la instrucción de salto).
- **BR:** Este módulo es un arreglo de 32 registros de 32 bits. Almacena los datos temporales y permite la lectura y escritura durante la ejecución de instrucciones.
- **SE:** Este módulo convierte un valor inmediato de menor tamaño en un valor de 32 bits, expandiéndolo según el valor de su bit más significativo para preservar el signo en operaciones de carga o salto. El extensor de signo toma un segmento de la instrucción y usa una señal de la unidad de control para generar un valor inmediato adecuado.
- **ALU:** La ALU realiza operaciones aritméticas (como suma o resta) y lógicas (como AND, OR) utilizando dos entradas: una del banco de registros y otra seleccionada por MUX2. La ALU es controlada por la señal de ALUControl, que determina la operación específica que debe realizar, en función del tipo de instrucción.
- **MUX2:** Selecciona entre dos posibles entradas para la ALU: Un valor leído del banco de registros o el valor inmediato generado por el extensor de signo. Esto permite flexibilidad para ejecutar instrucciones que usan un registro o un valor constante como entrada.
- **UC:** La unidad de control produce las señales de control que configuran los módulos en el datapath, tales como ALUControl para la ALU y las señales para la selección para los multiplexores. La UC determina la configuración adecuada para cada instrucción, a partir de los códigos f3, f7 y opcode de la instrucción.
- **mainDeco:** Se encarga de generar la señal de selección de operación para la ALU, en función del tipo de instrucción (códigos f3 y f7) y el opcode. Colabora con la unidad de control para establecer las configuraciones de operación específicas.
- **aluDeco:** Este módulo decodifica las señales de control y las traduce en instrucciones específicas que habilitan la ejecución en cada componente del datapath. Cuando interpreta las señales provenientes de la UC, permite operaciones aritméticas, lógicas o de control de flujo únicamente activando los módulos necesarios para la instrucción.
- **MUX3: (4x1):** Este módulo permite seleccionar entre múltiples fuentes de datos para determinar el valor final que será escrito o usado en el procesador. Tiene cuatro entradas y una salida, y usa señales de control para elegir cuál de las entradas debe pasar a la salida.

2.2. Módulo de memoria:

Este módulo combina tanto la memoria de instrucciones (IM) como la memoria de datos (DM) permitiendo al microprocesador almacenar y acceder a instrucciones y datos necesarios para ejecutar un programa en un mismo bloque. Este módulo se encarga de gestionar la comunicación entre el procesador y ambas memorias en un solo ciclo de reloj.

- **IM:** Este módulo se dedica a almacenar las instrucciones que el procesador va a ejecutar. Cada instrucción representa una operación que el procesador debe realizar. En cada ciclo de reloj, el procesador accede a esta memoria para obtener la instrucción correspondiente a la dirección actual del contador de programa (PC).
- **DM:** Esta memoria almacena los datos con los que trabaja el procesador durante la ejecución del programa. Puede que contenga variables, resultados de operaciones o cualquier otro tipo de datos que la CPU necesite leer o escribir. Es independiente a la memoria de instrucciones y permite acceder a ambas en paralelo en un ciclo de reloj.

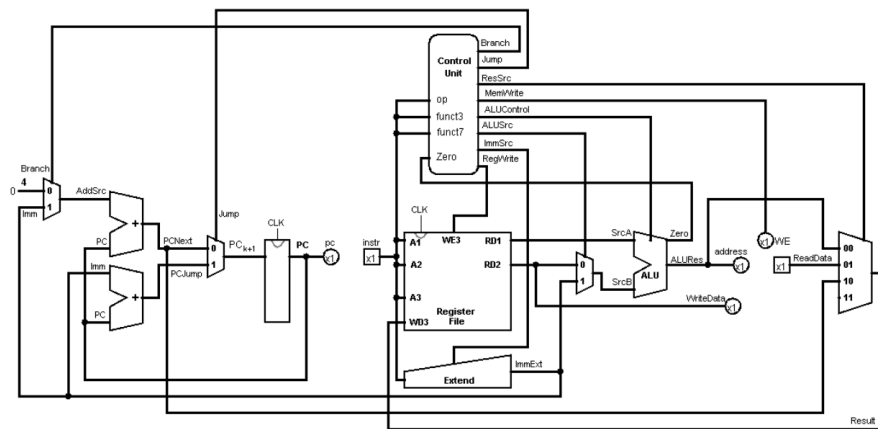


Figura 1: Diagrama de bloques del procesador RISC-V32I.

2.3. Módulo de Software

El diseño y la interconexión de los módulos son esenciales para garantizar un funcionamiento correcto y eficiente del procesador, ya que cada módulo debe coordinarse con precisión en el flujo de datos y control. En nuestro caso, hemos desarrollado un procesador monociclo de 32 bits orientado a operaciones con datos enteros, donde cada ciclo de reloj permite procesar datos de 32 bits, lo que asegura precisión y rapidez. Para verificar la correcta implementación, usaremos un código de prueba proporcionado por la cátedra. Este código realiza operaciones aritméticas y lógicas básicas (como suma y resta) entre valores inicializados en el banco de registros, comprobando que los módulos como la ALU y el banco de registros funcionen correctamente. Luego, el procesador ejecuta dos bucles de prueba: un bucle `while`, que duplica un valor hasta alcanzar un límite, evaluando así la integridad del flujo en condiciones iterativas, y un bucle `for`, que realiza sumas repetidas. Finalmente, los resultados se almacenan en la memoria de datos y se recargan en los registros, comprobando la precisión en el almacenamiento y recuperación de datos, y asegurando que el procesador pueda completar la secuencia completa de operaciones.

```
1  .text
2      # Inicializo tres variables
3      addi s0, zero, 3 # a = 3
4      addi s1, zero, 1 # b = 1
5      addi s2, zero, 16 # cte = 16
6      # operaciones lógicas aritméticas y slt
7      or t0, s0, s1 # c = 3
8      and t1, s0, s1 # d = 1
9      add t2, s0, s1 # e = 4
10     sub t3, s0, s1 # f = 2
11     sub t4, s1, s0 # g = 0xffffffffe = -2
12     slt t5, s0, s1 # h = 0
13     slt t6, s1, s0 # i = 1
14     slt t6, s1, t4, # j = 0
15     addi t0, zero, 1 # var = 1, variable de trabajo
16     addi t1, zero, 0 # cuenta = 0, un contador
17 while:
18     beq t0, s2, sal1 # si var == cte, sale del while
19     add t0, t0, t0 # var = var + var
20     addi t1, t1, 1 # cuenta = cuenta + 1
21     j while
22 sal1:
23     add s1, zero, zero # var = 0, $s0
24     addi t0, zero, 0 # indice = 0, $t0
25     addi t1, zero, 10 # veces = 10, $t1
26 for:
27     beq t0, t1, sal2 # if indice == veces, branch to done
28     add s1, s1, s0 # var = var + cte
29     addi t0, t0, 1 # incremento indice
30     j for
31 sal2:
32     sw s0, 0(zero) # guarda $s0 en registro 0
33     sw s1, 4(zero) # guarda $s1 en registro 4
34     sw s2, 8(zero) # guarda $s2 en registro 8
35     lw t0, 0(zero) # lee registro 0 en $t0
36     lw t1, 4(zero) # lee registro 4 en $t1
37     lw t2, 8(zero) # lee registro 8 en $t2
```

Figura 2: Código del programa proporcionado en ASM.

3. Pruebas

Para validar el correcto funcionamiento del sistema se han implementado módulos de test bench que permitieron simular y verificar cada parte del diseño. Estas pruebas incluyeron tanto evaluación de operaciones aritmético lógicas para poder comprobar el funcionamiento del banco de registros y la ALU, así como la ejecución de instrucciones de salto y bucles. Los resultados obtenidos fueron observados y comparados con los valores esperados, confirmando la correcta interacción entre los módulos. Además estas pruebas nos han servido para comprender con mayor profundidad el funcionamiento interno de cada componente, proporcionando información valiosa sobre el rendimiento y la estabilidad del diseño.

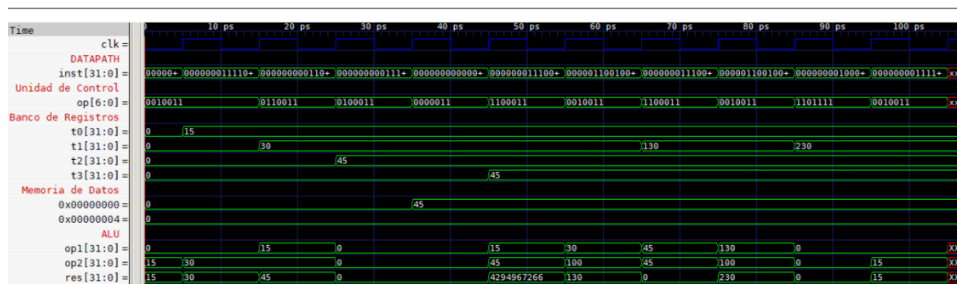


Figura 3: Test bench: Prueba completa.

4. Conclusiones

En este trabajo se han aplicado conocimientos de arquitectura RISC-V para implementar un procesador monociclo, lo que nos permitió observar cómo una estructura modular, organizada y eficiente puede formar una unidad de procesamiento funcional. La experiencia nos brindó una comprensión detallada del flujo de datos en la microarquitectura, asegurando la ejecución sincronizada de instrucciones en memoria. La naturaleza abierta de la arquitectura RISC también se destacó, mostrando su versatilidad para ser adaptada según las necesidades del usuario.

Referencias

- [1] M. Mano, *Diseño digital*. Los Angeles: Pearson Education, Inc., 2003.
- [2] Davit Patterson y Andrew Waterman, "Guía Práctica de RISC-V: El atlas de una arquitectura abierta.", 1era edición, Strawberry Canyon LLC San Francisco, Ca. USA 2018
- [3] L. Giovanini, "Apuntes de cátedra," FICH UNL, Cátedra de Organización de las computadoras, 2022.
- [4] J. E. Padula, "Apuntes de cátedra," FICH UNL, Cátedra de Organización de las computadoras, 2022.